



Hedieuhoa - Giới thiệu Shell. TinyShell

Nguyên lí hệ điều hành (Trường Đại học Mở Hà Nội)



messages.pdf_cover_qr_code_label

1. MỞ ĐẦU

1.1. Giới thiệu về Shell

Shell là một chương trình đặc biệt trong hệ điều hành, đóng vai trò là giao diện giữa người dùng và nhân hệ điều hành. Shell nhận lệnh từ người dùng, phân tích và thực thi các lệnh đó, sau đó trả về kết quả. Trong các hệ thống Unix-like, shell phổ biến bao gồm Bash, Zsh, Csh, trong khi Windows sử dụng Command Prompt (cmd.exe) hoặc PowerShell. Shell cung cấp môi trường để người dùng tương tác với hệ thống, quản lý tệp, chạy chương trình và quản lý tiến trình.

1.2. Mục tiêu của bài tập lớn

Bài tập lớn môn Nguyên lý hệ điều hành có mục tiêu thiết kế và triển khai một phiên bản shell đơn giản gọi là Tiny Shell. Chương trình này mô phỏng các chức năng cơ bản của một shell thực tế, bao gồm:

- Nhận và phân tích lệnh từ người dùng
- Thực thi các lệnh hệ thống
- Quản lý tiến trình foreground và background
- Cung cấp các lệnh tích hợp để điều khiển tiến trình
- Xử lý tín hiệu hệ thống

Thông qua việc triển khai Tiny Shell, sinh viên sẽ hiểu sâu hơn về cơ chế quản lý tiến trình, giao tiếp giữa các tiến trình, và cách hệ điều hành xử lý các lệnh từ người dùng.

1.3. Tính năng của Tiny Shell

Tiny Shell được phát triển với các tính năng chính sau:

- Giao diện dòng lệnh đơn giản: Hiển thị prompt "myShell>" để nhận lệnh từ người dùng
- Thực thi lệnh hệ thống: Cho phép chạy các chương trình ngoại vi như trong cmd.exe
- Quản lý tiến trình background: Hỗ trợ chạy tiến trình ở chế độ nền bằng cách thêm "&" vào cuối lệnh
- Lệnh tích hợp:
 - **help**: Hiển thị hướng dẫn sử dụng
 - **exit**: Thoát khỏi shell
 - **list**: Liệt kê các tiến trình background đang chạy
 - **kill <pid>**: Kết thúc tiến trình có PID chỉ định
 - **stop <pid>**: Tạm dừng tiến trình có PID chỉ định
 - **resume <pid>**: Tiếp tục tiến trình đã bị tạm dừng
 - **path**: Hiển thị biến môi trường PATH
 - **addpath <dir>**: Thêm thư mục vào biến PATH
- Xử lý tín hiệu: Hỗ trợ ngắt tiến trình foreground bằng tổ hợp phím Ctrl+C
- Đồng bộ hóa đa luồng: Sử dụng Critical Section để đảm bảo an toàn khi truy cập tài nguyên chia sẻ

2. CƠ SỞ LÝ THUYẾT

2.1. Khái niệm về Shell

Shell là một trình thông dịch dòng lệnh (command-line interpreter) cung cấp giao diện người dùng truyền thống. Shell hoạt động theo vòng lặp đọc-đánh giá-in (read-evaluate-print loop - REPL):

- **Đọc (Read)**: Đọc lệnh từ người dùng thông qua thiết bị nhập chuẩn (bàn phím)

- **Đánh giá (Evaluate):** Phân tích cú pháp lệnh, xác định xem đó là lệnh tích hợp hay chương trình ngoại vi
- **Thực thi (Execute):** Thực thi lệnh bằng cách tạo tiến trình mới hoặc gọi hàm tích hợp
- **In (Print):** Hiển thị kết quả hoặc thông báo lỗi ra thiết bị xuất chuẩn (màn hình)

Trong hệ điều hành Windows, shell tiêu chuẩn là cmd.exe, sử dụng các API của Windows để tạo và quản lý tiến trình. Tiny Shell được phát triển dựa trên các nguyên tắc này nhưng với chức năng đơn giản hơn.

2.2. Quản lý tiến trình trong Windows

- Tiến trình (process) là một thể hiện của một chương trình đang được thực thi. Trong Windows, mỗi tiến trình có:
 - Không gian địa chỉ riêng: Chứa mã, dữ liệu, heap và stack
 - Tài nguyên hệ thống: File, cửa sổ, cổng giao tiếp
 - Ít nhất một luồng thực thi (thread): Đơn vị cơ bản của việc sử dụng CPU

Windows cung cấp các API để quản lý tiến trình:

- **CreateProcess():** Tạo tiến trình mới và luồng chính của nó
- **TerminateProcess():** Kết thúc tiến trình
- **WaitForSingleObject():** Chờ tiến trình kết thúc
- **CloseHandle():** Giải phóng handle của tiến trình

Mỗi tiến trình được định danh bởi **Process ID (PID)** - một số duy nhất trong hệ thống.

2.3. Tiến trình foreground và background

Tiến trình foreground: Tiến trình đang tương tác trực tiếp với người dùng, nhận dữ liệu nhập từ bàn phím và hiển thị kết quả ra màn hình. Shell sẽ chờ tiến trình foreground kết thúc trước khi nhận lệnh tiếp theo.

Tiến trình background: Tiến trình chạy ở chế độ nền, không chiếm dụng shell. Người dùng có thể tiếp tục nhập lệnh khác trong khi tiến trình background vẫn đang chạy. Thường được đánh dấu bằng ký tự "&" ở cuối lệnh.

Trong Tiny Shell, tiến trình background được quản lý thông qua một danh sách (**bgList**) lưu trữ thông tin về các tiến trình đang chạy, bao gồm PID, trạng thái, và lệnh thực thi.

2.4. Điều khiển tiến trình: Dừng, Tiếp tục, Kết thúc

Tiny Shell cung cấp các lệnh để điều khiển tiến trình:

- **Kết thúc tiến trình (kill):** Sử dụng hàm **TerminateProcess()** để buộc tiến trình kết thúc ngay lập tức. Phương pháp này không an toàn vì tiến trình không có cơ hội dọn dẹp tài nguyên.
- **Tạm dừng tiến trình (stop):** Suspend tất cả các luồng của tiến trình bằng cách sử dụng **SuspendThread()**. Tiến trình vẫn tồn tại trong hệ thống nhưng không được cấp CPU.
- **Tiếp tục tiến trình (resume):** Resume các luồng đã bị suspend bằng **ResumeThread()**, cho phép tiến trình tiếp tục thực thi.

Việc điều khiển tiến trình đòi hỏi truy cập vào các luồng của tiến trình mục tiêu, được thực hiện thông qua ToolHelp API (**CreateToolhelp32Snapshot**, **Thread32First**, **Thread32Next**).

2.5. Biến môi trường PATH

Biến môi trường PATH là một danh sách các thư mục mà hệ điều hành tìm kiếm để thực thi các tệp chương trình. Khi người dùng nhập một lệnh, shell sẽ tìm kiếm tệp thực thi tương ứng trong các thư mục được chỉ định trong PATH.

Tiny Shell cung cấp hai lệnh để quản lý PATH:

- **path**: Hiển thị giá trị hiện tại của biến PATH
- **addpath <dir>**: Thêm một thư mục mới vào biến PATH

Việc thao tác với PATH được thực hiện thông qua các hàm **getenv()** và **SetEnvironmentVariableA()** của Windows API.

3. THIẾT KẾ HỆ THỐNG

3.1. Kiến trúc tổng quan

Tiny Shell được thiết kế theo kiến trúc đơn giản với các thành phần chính:

....

- **Command Parser**: Phân tích lệnh nhập từ người dùng, xác định xem đó là lệnh tích hợp hay chương trình ngoại vi, và phát hiện yêu cầu chạy background.
- **Process Manager**: Quản lý việc tạo tiến trình mới, theo dõi trạng thái của chúng, và cung cấp cơ chế để điều khiển tiến trình.
- **Built-in Commands**: Triển khai các lệnh tích hợp như help, exit, list, kill, stop, resume, path, addpath.
- **Signal Handler**: Xử lý các tín hiệu hệ thống như Ctrl+C để ngắt tiến trình foreground.
- **Background Process List**: Lưu trữ thông tin về các tiến trình đang chạy ở chế độ nền.
- **Environment Manager**: Quản lý biến môi trường, đặc biệt là PATH.

3.2. Cấu trúc dữ liệu

Tiny Shell sử dụng các cấu trúc dữ liệu chính sau:

```
cpp
1 enum ProcStatus { RUNNING, STOPPED, EXITED };
2
3 struct BGProc {
4     DWORD pid;           // Process ID
5     PROCESS_INFORMATION pi; // Process information
6     string cmdline;      // Command line that started the process
7     ProcStatus status;   // Current status of the process
8 };
9
10 vector<BGProc> bgList;    // List of background processes
11 PROCESS_INFORMATION fgProc; // Foreground process information
12 CRITICAL_SECTION cs;    // For thread synchronization
```

- **BGProc**: Cấu trúc lưu trữ thông tin về tiến trình background, bao gồm PID, thông tin tiến trình, dòng lệnh và trạng thái.
- **bgList**: Vector chứa danh sách các tiến trình background đang được quản lý.
- **fgProc**: Biến lưu trữ thông tin về tiến trình foreground hiện tại.
- **cs**: Critical Section để đảm bảo an toàn khi truy cập tài nguyên chia sẻ từ nhiều luồng.

3.3. Luồng xử lý chính

Tiny Shell hoạt động theo vòng lặp chính như sau:

...

Khởi tạo shell (Initialize Shell): Thiết lập Critical Section và đăng ký handler cho tín hiệu Ctrl+C.

Hiển thị prompt (Display Prompt): Hiển thị "myShell>" để người dùng nhập lệnh.

Đọc lệnh (Read Command): Nhận lệnh từ người dùng thông qua hàm `fgets()`.

Phân tích lệnh (Parse Command): Tách lệnh thành các thành phần, xác định xem có yêu cầu background không (ký tự '&').

Kiểm tra lệnh tích hợp (Check for Built-in Commands): Nếu là lệnh tích hợp, thực thi ngay.

Tạo tiến trình (Create Process): Nếu là chương trình ngoại vi, tạo tiến trình mới bằng `CreateProcessA()`.

Xử lý tiến trình (Wait/Track): Với tiến trình foreground, chờ nó kết thúc. Với tiến trình background, thêm vào danh sách quản lý.

Lặp lại (Continue loop): Quay lại bước 2 để tiếp tục nhận lệnh tiếp theo.

3.4. Các module chức năng

Tiny Shell được chia thành các module chức năng chính:

1. **Module Quản lý Tiến trình:**
 - `add_bg_proc()`: Thêm tiến trình vào danh sách background
 - `remove_exited_bg()`: Loại bỏ các tiến trình đã kết thúc khỏi danh sách
 - `list_procs()`: Hiển thị danh sách tiến trình background
 - `kill_proc()`: Kết thúc tiến trình có PID chỉ định
 - `stop_proc()`: Tạm dừng tiến trình có PID chỉ định
 - `resume_proc()`: Tiếp tục tiến trình đã bị tạm dừng
2. **Module Xử lý Lệnh Tích hợp:**
 - `print_help()`: Hiển thị hướng dẫn sử dụngXử lý các lệnh: exit, list, kill, stop, resume, path, addpath
3. **Module Xử lý Tín hiệu:**
 - `consoleHandler()`: Xử lý tín hiệu Ctrl+C để ngắt tiến trình foreground
4. **Module Phân tích Lệnh:**

Phân tích đầu vào để xác định lệnh tích hợp hay chương trình ngoại vi
Phát hiện yêu cầu chạy background (ký tự '&')
5. **Module Quản lý Môi trường:**

Hiển thị và cập nhật biến môi trường PATH

4. CHI TIẾT TRIỂN KHAI

4.1. Khởi tạo hệ thống

Hàm `main()` là điểm khởi đầu của Tiny Shell, thực hiện các bước khởi tạo cần thiết:

cpp

```
1 int main() {
2     InitializeCriticalSection(&cs); // Khởi tạo Critical Section
3     SetConsoleCtrlHandler(consoleHandler, TRUE); // Đăng ký handler cho Ctrl+C
4     run_shell(); // Bắt đầu vòng lặp chính
5     DeleteCriticalSection(&cs); // Giải phóng Critical Section
6     printf("Bye.\n");
7     return 0;
8 }
```

- **Critical Section:** Được khởi tạo để bảo vệ các tài nguyên chia sẻ (như danh sách tiến trình background) khỏi xung đột khi truy cập đồng thời từ nhiều luồng.
- **Console Control Handler:** Đăng ký hàm `consoleHandler` để xử lý sự kiện Ctrl+C, cho phép ngắt tiến trình foreground một cách an toàn.
- **Shell Loop:** Hàm `run_shell()` chứa vòng lặp chính của shell, nơi nhận và xử lý lệnh từ người dùng.

4.2. Phân tích lệnh

Tiny Shell phân tích lệnh nhập từ người dùng thông qua các bước sau:

cpp

```
1 char line[MAX_CMD];
2 fgets(line, sizeof(line), stdin); // Đọc lệnh từ người dùng
3
4 // Loại bỏ ký tự xuống dòng
5 size_t len = strlen(line);
6 while (len && (line[len-1] == '\n' || line[len-1] == '\r')) {
7     line[--len] = 0;
8 }
9
10 // Kiểm tra yêu cầu background
11 bool background = false;
12 if (!buf.empty() && buf.back() == '&') {
13     background = true;
14     buf.pop_back(); // Loại bỏ ký tự '&'
15     while (!buf.empty() && buf.back() == ' ') buf.pop_back(); // Loại bỏ khoảng trắng
16 }
17
18 // Tách lệnh thành các token
19 istream iss(buf);
20 string tok;
21 iss >> tok; // Lấy từ đầu tiên (tên lệnh)
```

- **Đọc lệnh:** Sử dụng `fgets()` để đọc lệnh từ stdin vào bộ đệm `line`.
- **Loại bỏ ký tự xuống dòng:** Xóa các ký tự '\n' và '\r' ở cuối chuỗi.
- **Phát hiện background:** Kiểm tra ký tự '&' ở cuối lệnh để xác định yêu cầu chạy background.

- **Tách token:** Sử dụng `istringstream` để tách lệnh thành các thành phần, từ đầu tiên là tên lệnh.

4.3. Xử lý lệnh tích hợp

Tiny Shell hỗ trợ các lệnh tích hợp được xử lý trực tiếp mà không tạo tiến trình mới:

```
cpp
1  if (_stricmp(tok.c_str(), "exit") == 0) {
2      // xử lý lệnh exit
3  } else if (_stricmp(tok.c_str(), "help") == 0) {
4      print_help();
5  } else if (_stricmp(tok.c_str(), "list") == 0) {
6      list_procs();
7  } else if (_stricmp(tok.c_str(), "kill") == 0) {
8      DWORD pid; iss >> pid;
9      if (kill_proc(pid) == 0)
10         printf("Killed %u\n", (unsigned)pid);
11     else
12         printf("kill failed for %u\n", (unsigned)pid);
13 } // ... các lệnh khác
```

- **exit:** Kết thúc tất cả tiến trình background và thoát shell.
- **help:** Hiển thị hướng dẫn sử dụng thông qua hàm `print_help()`.
- **list:** Liệt kê các tiến trình background đang chạy bằng `list_procs()`.
- **kill:** Kết thúc tiến trình có PID chỉ định thông qua `kill_proc()`.
- **stop:** Tạm dừng tiến trình bằng `stop_proc()`.
- **resume:** Tiếp tục tiến trình đã bị tạm dừng bằng `resume_proc()`.
- **path:** Hiển thị biến môi trường PATH.
- **addpath:** Thêm thư mục vào biến PATH.

Hàm `_stricmp()` được sử dụng để so sánh chuỗi không phân biệt chữ hoa/thường.

4.4. Tạo tiến trình mới

Đối với các lệnh không phải là lệnh tích hợp, Tiny Shell tạo tiến trình mới để thực thi:

cpp

```
1 // Xây dựng dòng lệnh
2 string cmdline(line);
3
4 // Nếu là file .bat, chạy qua cmd.exe
5 string finalCmd;
6 if (is_bat(tok)) {
7     finalCmd = "cmd.exe /c " + cmdline;
8 } else {
9     finalCmd = cmdline;
10 }
11
12 // Thiết lập cấu trúc STARTUPINFO và PROCESS_INFORMATION
13 STARTUPINFOA si{};
14 PROCESS_INFORMATION pi{};
15 si.cb = sizeof(si);
16
17 // Tạo tiến trình
18 BOOL ok = CreateProcessA(
19     NULL,           // Tên module (NULL = sử dụng dòng lệnh)
20     finalCmd.data(), // Dòng lệnh
21     NULL,           // Security attributes (process)
22     NULL,           // Security attributes (thread)
23     TRUE,           // Kế thừa handle
24     CREATE_NEW_CONSOLE, // Tạo console mới
25     NULL,           // Environment block
26     NULL,           // Thư mục làm việc
27     &si,             // STARTUPINFO
28     &pi              // PROCESS_INFORMATION
29 );
```

- **Xây dựng dòng lệnh:** Giữ nguyên dòng lệnh nhập từ người dùng.
- **Xử lý file .bat:** Nếu lệnh là file batch (.bat), chạy thông qua `cmd.exe /c`.
- **Cấu trúc STARTUPINFO:** Chứa thông tin về cách hiển thị cửa sổ tiến trình.
- **Cấu trúc PROCESS_INFORMATION:** Nhận thông tin về tiến trình mới được tạo (PID, handle).
- **CreateProcessA:** API Windows để tạo tiến trình mới với các tham số đã cấu hình.

4.5. Quản lý tiến trình background

Tiến trình background được quản lý thông qua danh sách `bgList`:

cpp

```
1 void add_bg_proc(PROCESS_INFORMATION *pi, const string &cmdline) {
2     EnterCriticalSection(&cs);
3     if (bgList.size() < MAX_BG) {
4         BGProc proc;
5         proc.pid = pi->dwProcessId;
6         proc.pi = *pi;
7         proc.cmdline = cmdline;
8         proc.status = RUNNING;
9         bgList.push_back(proc);
10    } else {
11        TerminateProcess(pi->hProcess, 0);
12        CloseHandle(pi->hProcess);
13        CloseHandle(pi->hThread);
14        printf("Too many background processes. Killed new one.\n");
15    }
16    LeaveCriticalSection(&cs);
17 }
```

- **Thêm tiến trình:** Hàm `add_bg_proc()` thêm tiến trình vào `bgList` nếu còn chỗ.
- **Giới hạn số lượng:** Nếu đạt giới hạn (`MAX_BG`), tiến trình mới sẽ bị kết thúc.
- **Bảo vệ đồng bộ:** Sử dụng `EnterCriticalSection` và `LeaveCriticalSection` để đảm bảo an toàn khi truy cập danh sách từ nhiều luồng.
- **Thông tin tiến trình:** Lưu trữ PID, thông tin tiến trình, dòng lệnh và trạng thái.

Để loại bỏ các tiến trình đã kết thúc:

cpp

```
1 void remove_exited_bg() {
2     EnterCriticalSection(&cs);
3     for (size_t i = 0; i < bgList.size(); ) {
4         DWORD res = WaitForSingleObject(bgList[i].pi.hProcess, 0);
5         if (res == WAIT_OBJECT_0) {
6             CloseHandle(bgList[i].pi.hProcess);
7             CloseHandle(bgList[i].pi.hThread);
8             bgList.erase(bgList.begin() + i);
9         } else {
10             ++i;
11         }
12     }
13     LeaveCriticalSection(&cs);
14 }
```

- **Kiểm tra trạng thái:** Sử dụng `WaitForSingleObject` với thời gian chờ 0 để kiểm tra tiến trình đã kết thúc chưa.

- **Dọn dẹp tài nguyên:** Đóng handle của tiến trình và luồng khi chúng đã kết thúc.
- **Cập nhật danh sách:** Loại bỏ tiến trình đã kết thúc khỏi danh sách.

4.6. Xử lý tín hiệu điều khiển

Tiny Shell xử lý tín hiệu Ctrl+C để ngắt tiến trình foreground:

```
cpp
1  BOOL WINAPI consoleHandler(DWORD ctrlType) {
2      if (ctrlType == CTRL_C_EVENT) {
3          EnterCriticalSection(&cs);
4          if (fgProc.hProcess) {
5              TerminateProcess(fgProc.hProcess, 1);
6          }
7          LeaveCriticalSection(&cs);
8          return TRUE;
9      }
10     return FALSE;
```

- **Đăng ký handler:** Handler được đăng ký trong hàm `main()` thông qua `SetConsoleCtrlHandler`.
- **Xử lý Ctrl+C:** Khi nhận tín hiệu Ctrl+C, kiểm tra nếu có tiến trình foreground đang chạy thì kết thúc nó.
- **Bảo vệ đồng bộ:** Sử dụng Critical Section để đảm bảo an toàn khi truy cập biến `fgProc`.
- **Trả về TRUE:** Cho biết đã xử lý tín hiệu, ngăn hệ thống xử lý tiếp.

4.7. Đồng bộ hóa đa luồng

Tiny Shell sử dụng Critical Section để đồng bộ hóa truy cập vào tài nguyên chia sẻ:

```
cpp
1  CRITICAL_SECTION cs;
2
3  // Khởi tạo
4  InitializeCriticalSection(&cs);
5
6  // Trước khi truy cập tài nguyên chia sẻ
7  EnterCriticalSection(&cs);
8
9  // Thao tác với tài nguyên chia sẻ
10 // ...
11
12 // Sau khi hoàn thành
13 LeaveCriticalSection(&cs);
14
15 // Giải phóng khi kết thúc
16 DeleteCriticalSection(&cs);
```

- **Mục đích:** Ngăn ngừa xung đột khi nhiều luồng truy cập đồng thời vào danh sách tiến trình background hoặc biến tiến trình foreground.
- **Cách hoạt động:** Critical Section đảm bảo chỉ một luồng có thể truy cập vào vùng mã được bảo vệ tại một thời điểm.
- **Hiệu suất:** Critical Section là cơ chế đồng bộ hóa nhẹ nhàng, phù hợp cho các ứng dụng đơn luồng như Tiny Shell.

5. KẾT QUẢ THỰC NGHIỆM

5.1. Môi trường thử nghiệm

Tiny Shell được thử nghiệm trên môi trường sau:

- **Hệ điều hành:** Windows 10 Pro 64-bit
- **Bộ vi xử lý:** Intel Core i7-8650U @ 1.90GHz
- **RAM:** 16GB
- **Bộ biên dịch:** Microsoft Visual C++ 2019
- **IDE:** Visual Studio 2019

Các lệnh và chương trình được sử dụng để thử nghiệm:

- Lệnh hệ thống: **dir**, **ping**, **notepad**
- File batch: **test.bat** (đơn giản)
- Chương trình C++ tự viết: **testprog.exe**

5.2. Kiểm thử chức năng cơ bản

...

6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Tổng kết kết quả đạt được

Qua quá trình thực hiện bài tập lớn, chúng em đã triển khai thành công Tiny Shell - một phiên bản shell đơn giản trên Windows với các tính năng chính:

- **Giao diện dòng lệnh:** Cung cấp prompt "myShell>" để nhận lệnh từ người dùng.
- **Thực thi lệnh hệ thống:** Cho phép chạy các chương trình ngoại vi như trong cmd.exe.
- **Quản lý tiến trình:** Hỗ trợ chạy tiến trình foreground và background, điều khiển tiến trình (kill, stop, resume).
- **Lệnh tích hợp:** Cung cấp các lệnh tiện ích như help, exit, list, path, addpath.
- **Xử lý tín hiệu:** Hỗ trợ ngắt tiến trình foreground bằng Ctrl+C.
- **Đồng bộ hóa:** Sử dụng Critical Section để đảm bảo an toàn khi truy cập tài nguyên chia sẻ.

Tiny Shell đã được kiểm thử kỹ lưỡng trên nhiều tình huống khác nhau và cho kết quả hoạt động ổn định, đáp ứng các yêu cầu cơ bản của bài tập.

6.2. Hạn chế của hệ thống

Mặc dù đạt được các mục tiêu cơ bản, Tiny Shell vẫn còn một số hạn chế:

- **Khả năng mở rộng:** Thiếu cơ chế plugin hoặc module để mở rộng chức năng.
- **Xử lý lỗi:** Một số trường hợp lỗi chưa được xử lý triệt để (ví dụ: khi nhập PID không tồn tại).
- **Hỗ trợ lệnh:** Chỉ hỗ trợ các lệnh đơn giản, không hỗ trợ pipe, redirect, hay script.
- **Giao diện:** Giao diện dòng lệnh đơn giản, không có tính năng tự động hoàn thành lệnh hay lịch sử lệnh.
- **Tính di động:** Chỉ hoạt động trên Windows, không thể chạy trên các hệ điều hành khác.

6.3. Hướng phát triển trong tương lai

Dựa trên kết quả đạt được và các hạn chế hiện tại, chúng em đề xuất một số hướng phát triển cho Tiny Shell trong tương lai:

Bổ sung tính năng nâng cao:

- Hỗ trợ pipe (|) và redirect (>, <, >>)
- Thêm lịch sử lệnh (lên/xuống để chọn lệnh đã dùng)
- Tự động hoàn thành lệnh (tab completion)

Cải thiện giao diện:

- Hỗ trợ màu sắc cho đầu ra
- Hiển thị thông tin chi tiết hơn về tiến trình
- Cung cấp chế độ tương tác nâng cao

Tăng tính ổn định:

- Xử lý lỗi tốt hơn
- Thêm cơ chế phục hồi khi gặp lỗi
- Tối ưu hóa việc quản lý tài nguyên

Mở rộng khả năng:

- Hỗ trợ script shell đơn giản
- Thêm cơ chế plugin để mở rộng
- Phát triển phiên bản đa nền tảng (Linux, macOS)

Nâng cao hiệu năng:

- Tối ưu hóa việc tạo và quản lý tiến trình
- Giảm thời gian phản hồi
- Cải thiện sử dụng bộ nhớ

Tiny Shell là một bước khởi đầu tốt để hiểu về cơ chế hoạt động của shell trong hệ điều hành. Việc phát triển thêm các tính năng sẽ biến nó thành một công cụ hữu ích hơn cho người dùng và đồng thời giúp sinh viên hiểu sâu hơn về nguyên lý hệ điều hành.